Part 3: Asynchronous GRPO

Setup and learning outcome

I ran p3_number_search_async unchanged, independently from the base Qwen/Qwen3.5-4B model, with LoRA rank 32, PPO, learning rate 10^{-5}, and seed 11768. Eight rollout workers feed a queue of capacity 16; each learner iteration accepts eight groups of eight rollouts, with maximum staleness 2. The server splits two H100s between training and sampling. Evaluation uses the first 50 validation tasks, four rollouts each, at baseline and every ten iterations. The comparisons below use the completed formal p2b and p3 runs; probes are excluded. Each configuration was run once.

Metric	p2b: synchronous	p3: asynchronous
Initial eval/success_rate	34.5%	31.5%
Final eval/success_rate	99.0%	99.5%
Final eval/pass@4	100%	100%
Iterations / optimizer updates	50 / 34	50 / 40
Skipped iterations	16	10
Mean train/step_time_s	38.03 s	32.13 s
Median train/step_time_s	27.92 s	27.95 s
Sum of iteration times	31.69 min	26.78 min
Baseline result to final evaluation	32.89 min	28.07 min
Time-weighted learner-wait fraction	66.3%	35.7%

p3's validation success rises from 31.5% to 44.0%, 80.0%, 98.5%, 99.0%, and 99.5% at iterations 10 through 50. It initially learns more slowly per iteration than p2b, which reaches 98% at iteration 20, but both finish near 100%. The final difference is one successful episode among 200, so this single-run comparison does not establish a quality advantage.

Wall-clock comparison

Using all 50 logged train/step_time_s values, including skipped iterations, the speedup is 38.0287 / 32.1329 = 1.1835, or 15.5% less time per iteration. The median is essentially unchanged: the speedup describes average and total time, not every individual iteration. The timing intervals start after the baseline result and run between training records. They include first-iteration and checkpoint overhead, and synchronous periodic evaluations enter the following iteration's interval. They exclude cold start, baseline evaluation, and the final evaluation drain; the latter adds 71.76 s for p2b and 77.46 s for p3.

The time-weighted learner-wait fraction, sum(learner_wait_s) / sum(step_time_s), falls from 66.3% to 35.7%, consistent with overlapping rollouts and background evaluation with optimization. This residual measures time outside updates and sampler saves, not GPU utilization. It is not the arithmetic mean of per-iteration fractions, which would overweight near-instant skipped iterations. The comparison also includes different GPU roles (two colocated training/sampling GPUs versus one GPU for each role), task order and realized samples, 34 versus 40 actual updates, and 1,200 versus 1,720 retained sequences. It therefore measures the provided complete systems rather than an isolated scheduling optimization.

Stale data and completion

p3 accepted 400 groups and discarded 71 stale groups: 15.1% of the 471 dequeued groups, not of all generated groups. Queue leftovers and in-flight groups at shutdown are outside these counters. Importance weighting corrects the accepted older-policy data, with the ratio tails discussed in Figures 2 and 3. There is no observed final-quality penalty relative to p2b, although learning per iteration differs. All abandoned-episode counts are zero. Ten all-success batches carry no GRPO signal and skip updating, so 50 iterations produce version 40; the final sampler is v40 and the last resumable state is iteration 49 / update 40, before the skipped final iteration.

Early, middle, and late training behavior

The following three real training rollouts come from runs/p3_number_search_async_trajectories/. Their logged step identifies the completion interval, while policy_version identifies the snapshot that generated them. H, L, and C below mean too high, too low, and correct. These are illustrative examples on different tasks, not a controlled estimate of average search efficiency.

Early: step 5, policy version 4

Task number_search.train.65 has range [301, 697], target 471, and a nine-guess budget. Source: step_000005 / train_number_search.train.65_rollout0.jsonl.

Guesses: 500 H → 350 L → 450 L → 480 H → 440 L → 460 L → 470 L → 475 H → 470 L. The episode exhausts all nine guesses and receives reward 0. After learning that the target exceeds 450, the agent tries 440; later it repeats 470 despite already receiving “too low” for that value. These are strategically inconsistent guesses, not parser or argument errors.

Middle: step 25, policy version 24

Task number_search.train.272 has range [142, 669], target 242, and a ten-guess budget. Source: step_000025 / train_number_search.train.272_rollout0.jsonl.

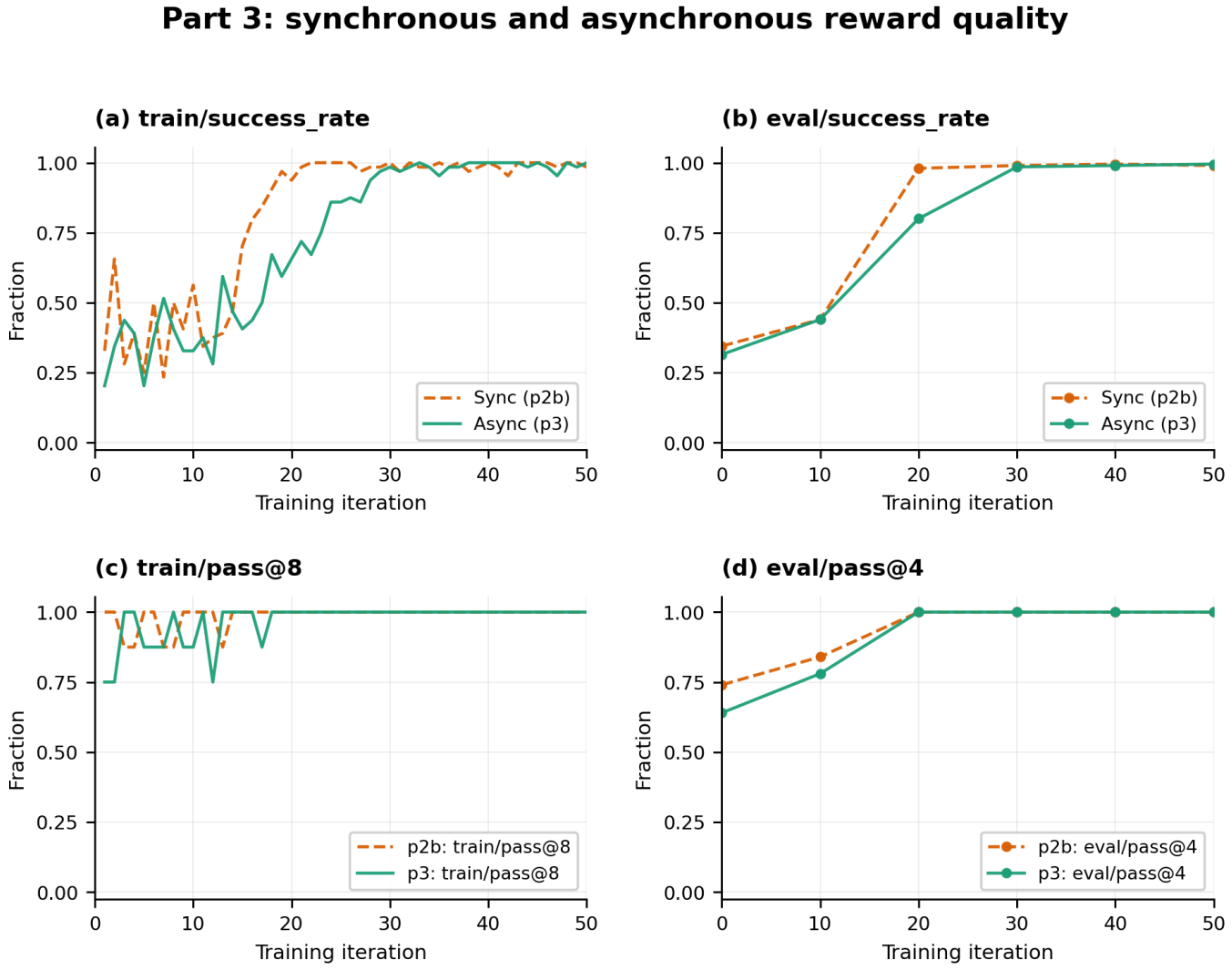
Guesses: 405 H → 283 H → 221 L → 252 H → 236 L → 244 H → 240 L → 242 C. It succeeds in eight guesses, receiving reward 1. Every guess lies within the interval consistent with earlier feedback, although the second and third probes differ from that interval's midpoint.

Late: step 45, policy version 36

Task number_search.train.428 has range [538, 892], target 862, and a nine-guess budget. Source: step_000045 / train_number_search.train.428_rollout0.jsonl.

Guesses: 715 L → 804 L → 848 L → 871 H → 859 L → 865 H → 862 C. It succeeds in seven guesses, receiving reward 1. Every action chooses a floor or ceiling midpoint of the integer interval still consistent with all previous feedback.

Two concrete changes appear in these examples: better respect for feedback-derived bounds, and more systematic bisection. The number of guesses outside the feasible interval falls from two in the early example to zero in the middle and late examples. Midpoint choices account for 2/9, 6/8, and 7/7 actions, respectively. Bounds are updated using all earlier feedback, so an inconsistent guess never widens the feasible interval. These examples support the behavioral interpretation alongside the held-out reward improvement, while the differing tasks prevent attributing their exact guess counts solely to training.



Unsmoothed formal runs; iteration 0 is the evaluation baseline.

Figure 1. Reward quality versus training iteration. Both formal runs show train/success_rate, eval/success_rate, train/pass@8, and eval/pass@4, without smoothing. Training success improves markedly in both runs and later approaches 1. The higher training pass@8 curves measure whether at least one of eight attempts succeeds, rather than the average success of an episode; evaluation pass@4 uses a different k. p3 reaches evaluation pass@4 = 1 at iteration 20, while its per-episode evaluation success continues rising to 99.5%. The slower initial rise in p3's evaluation success relative to p2b is visible despite their similar final results. Train rewards describe accepted pre-update rollouts, potentially from older versions; asynchronous evaluations use the snapshot captured at the labeled iteration and may finish later. Iteration-50 evaluation uses p3 version 40 because skipped iterations do not advance the policy.

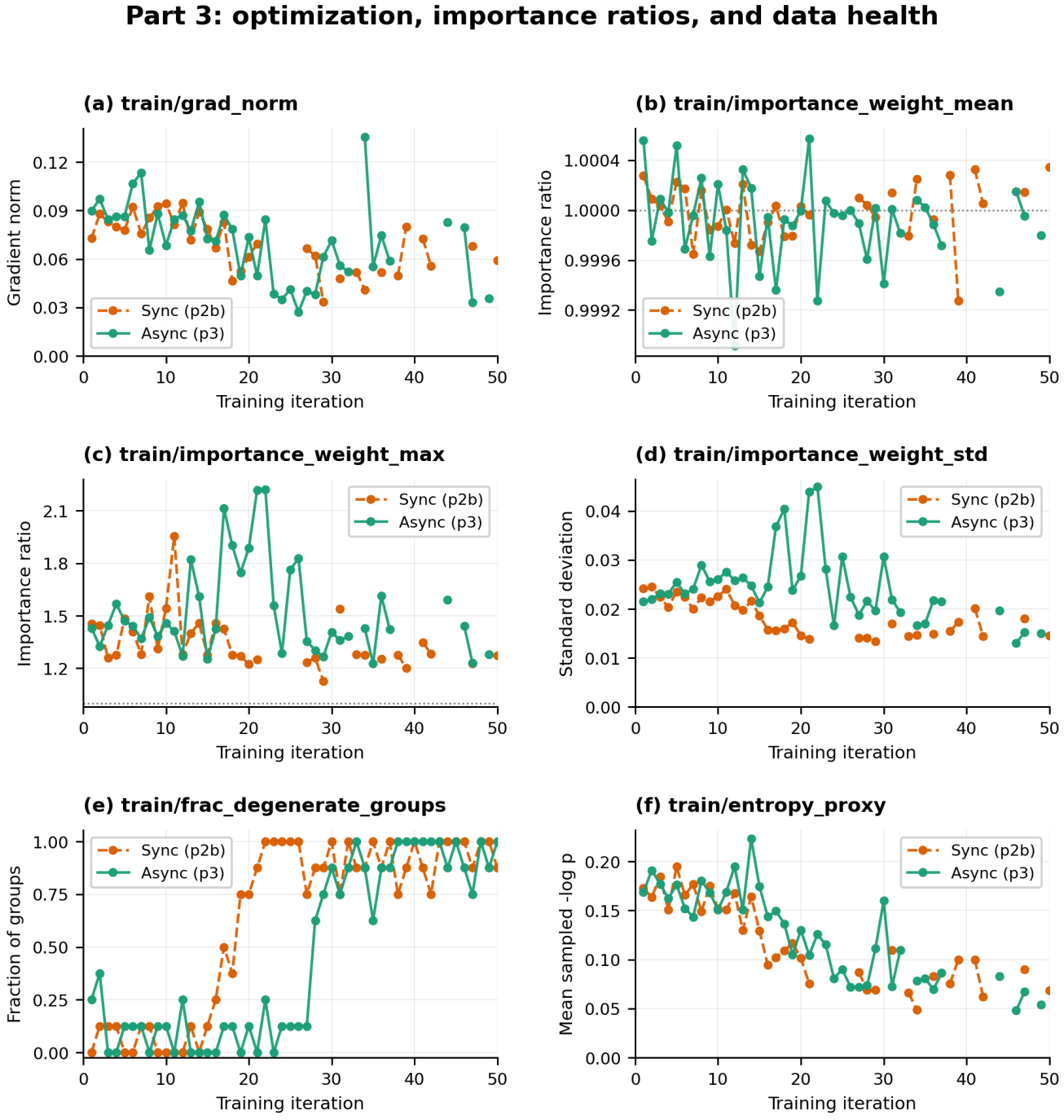
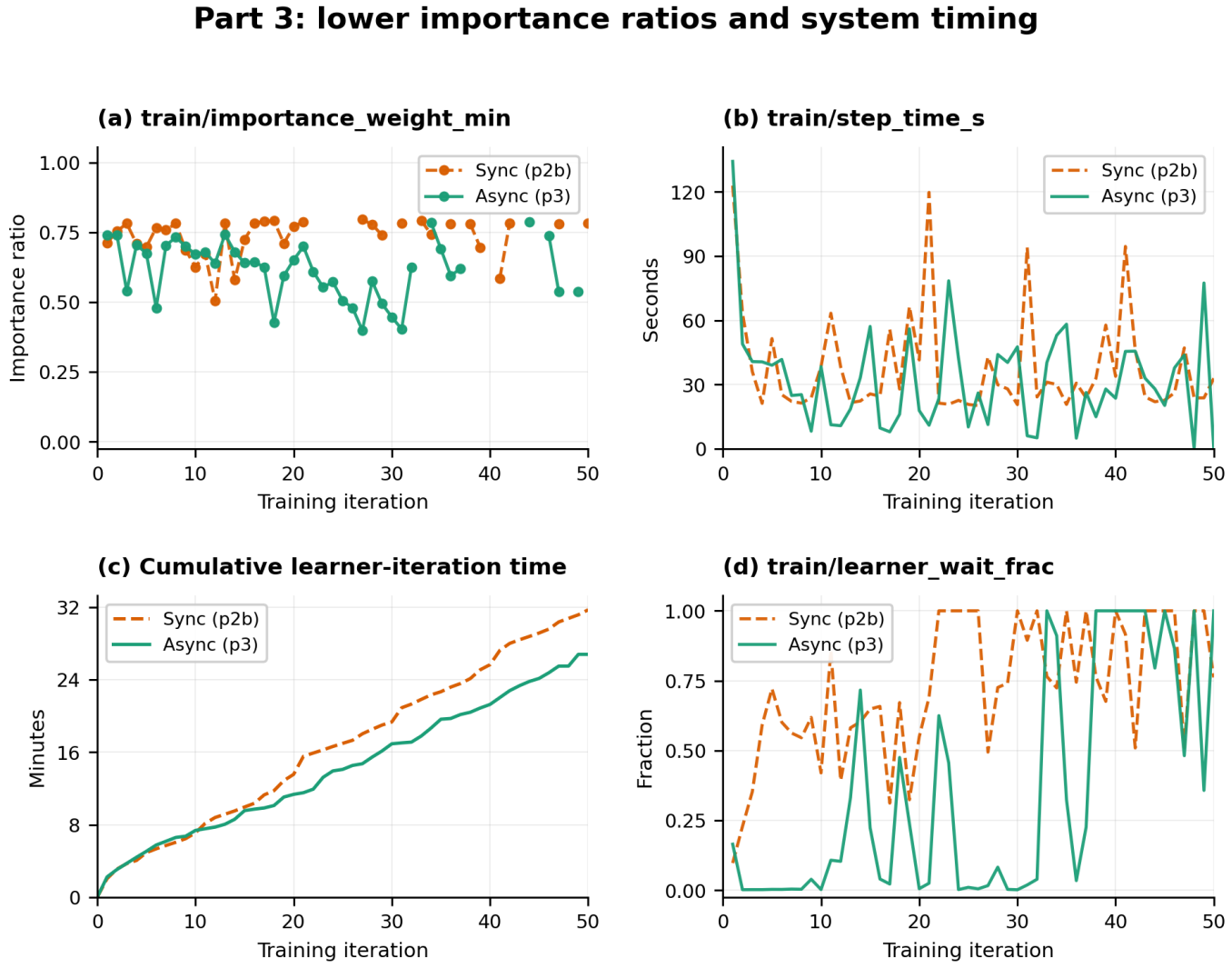


Figure 2. Optimization, importance ratios, and data health. Gradient norms remain finite: 0.0335–0.0946 for p2b and 0.0272–0.1355 for p3. The per-update IW means remain near 1 (p2b: 0.999274–1.000343; p3: 0.998915–1.000574), but p3 has a wider tail: its maximum reaches 2.219 versus 1.954, and its highest standard deviation is 0.04494 versus 0.02450. The minima in Figure 3 complete this comparison; the mean alone would hide these differences. These action-token statistics are raw ratios, so values outside PPO's clipping interval are expected and do not mean clipping failed. Degenerate fractions rise with uniformly successful groups; the 16 p2b and 10 p3 fully degenerate iterations skip updates and retain gaps in gradient, IW, and entropy curves. The action-token surprise proxy decreases from 0.173 to 0.069 in p2b and from 0.169 to 0.054 in p3. It describes retained nondegenerate groups and predictable format tokens as well as decisions, so its decline alone does not establish collapse.

Part 3 · Diagnostic plots (3/3)



Timing includes all 50 iterations; cumulative time excludes final evaluation drain.

Figure 3. Lower importance ratios and timing. The minimum IW falls as low as 0.504 in p2b and 0.398 in p3, complementing p3's wider upper tail and standard deviation in Figure 2. This is measurable sampling/training mismatch without a runaway tail in these runs. Raw step times retain long iterations and very short skipped iterations; their similar medians do not contradict the lower asynchronous mean. Cumulative time sums all 50 iteration intervals and ends at 31.69 versus 26.78 minutes, before final evaluation drain. The plotted learner-wait fraction is the raw per-iteration value; it can equal 1 on a skipped iteration that takes almost no wall time. The report table therefore uses time-weighted fractions, 66.3% versus 35.7%, to summarize reduced waiting. Overlap and background evaluation are consistent with this reduction, while the differences in GPU roles, actual updates, and retained batches limit causal attribution.